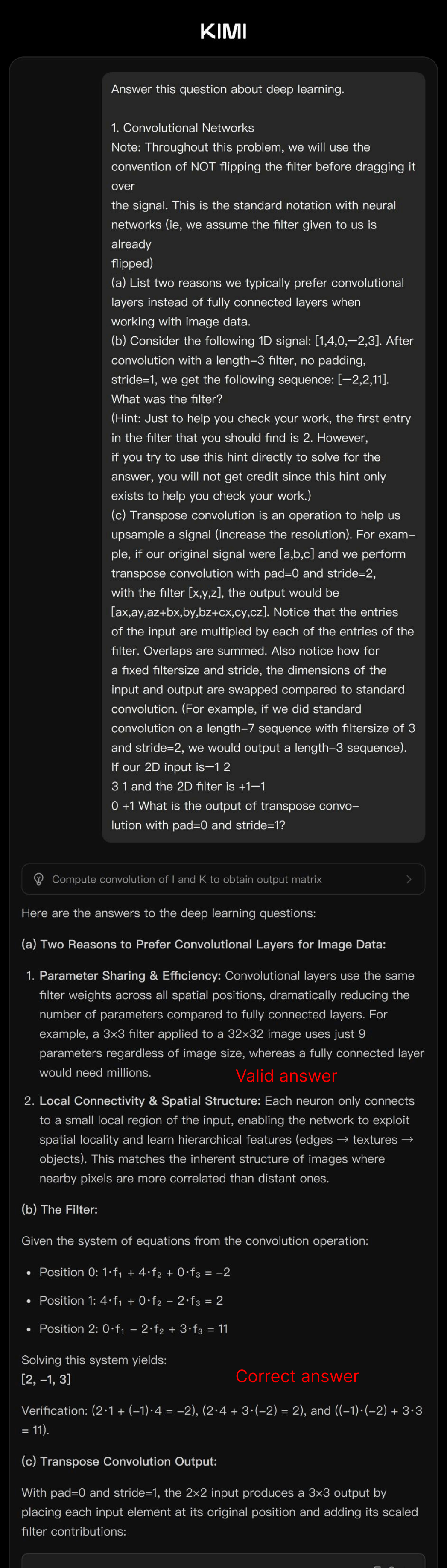


KIMI

Answer this question about deep learning.

1. Convolutional Networks
Note: Throughout this problem, we will use the convention of NOT flipping the filter before dragging it over the signal. This is the standard notation with neural networks (ie, we assume the filter given to us is already flipped)
(a) List two reasons we typically prefer convolutional layers instead of fully connected layers when working with image data.
(b) Consider the following 1D signal: [1,4,0,−2,3]. After convolution with a length-3 filter, no padding, stride=1, we get the following sequence: [−2,2,11]. What was the filter?
(Hint: Just to help you check your work, the first entry in the filter that you should find is 2. However, if you try to use this hint directly to solve for the answer, you will not get credit since this hint only exists to help you check your work.)
(c) Transpose convolution is an operation to help us upsample a signal (increase the resolution). For example, if our original signal were [a,b,c] and we perform transpose convolution with pad=0 and stride=2, with the filter [x,y,z], the output would be [ax,ay,az+bx,by,bz+cx,cy,cz]. Notice that the entries of the input are multiplied by each of the entries of the filter. Overlaps are summed. Also notice how for a fixed filtersize and stride, the dimensions of the input and output are swapped compared to standard convolution. (For example, if we did standard convolution on a length-7 sequence with filtersize of 3 and stride=2, we would output a length-3 sequence).
If our 2D input is−1 2
3 1 and the 2D filter is +1−1
0 +1 What is the output of transpose convolution with pad=0 and stride=1?

Compute convolution of I and K to obtain output matrix

Here are the answers to the deep learning questions:

(a) Two Reasons to Prefer Convolutional Layers for Image Data:

1. **Parameter Sharing & Efficiency:** Convolutional layers use the same filter weights across all spatial positions, dramatically reducing the number of parameters compared to fully connected layers. For example, a 3×3 filter applied to a 32×32 image uses just 9 parameters regardless of image size, whereas a fully connected layer would need millions.

Valid answer

2. **Local Connectivity & Spatial Structure:** Each neuron only connects to a small local region of the input, enabling the network to exploit spatial locality and learn hierarchical features (edges → textures → objects). This matches the inherent structure of images where nearby pixels are more correlated than distant ones.

(b) The Filter:

Given the system of equations from the convolution operation:

- Position 0: $1\cdot f_1 + 4\cdot f_2 + 0\cdot f_3 = -2$
- Position 1: $4\cdot f_1 + 0\cdot f_2 - 2\cdot f_3 = 2$
- Position 2: $0\cdot f_1 - 2\cdot f_2 + 3\cdot f_3 = 11$

Solving this system yields:
[2, −1, 3]

Correct answer

Verification: $(2\cdot 1 + (-1)\cdot 4 = -2)$, $(2\cdot 4 + 3\cdot(-2) = 2)$, and $((-1)\cdot(-2) + 3\cdot 3 = 11)$.

(c) Transpose Convolution Output:

With pad=0 and stride=1, the 2×2 input produces a 3×3 output by placing each input element at its original position and adding its scaled filter contributions:

Copy

```
-1   3  -2
 3  -3   1
 0   3   1
```

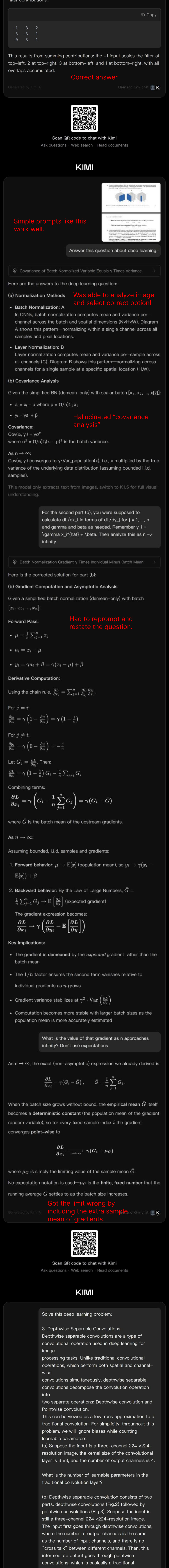
This results from summing contributions: the −1 input scales the filter at top-left, 2 at top-right, 3 at bottom-left, and 1 at bottom-right, with all overlaps accumulated.

Correct answer

Generated by Kimi AI

User and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents



KIMI

Simple prompts like this work well.

Answer this question about deep learning.

Covariance of Batch Normalized Variable Equals γ Times Variance

Here are the answers to the deep learning question:

Was able to analyze image and select correct option!

(a) Normalization Methods

- **Batch Normalization: A**
In CNNs, batch normalization computes mean and variance per-channel across the batch and spatial dimensions (N×H×W). Diagram A shows this pattern—normalizing along within a single channel across all samples and pixel locations.
- **Layer Normalization: B**
Layer normalization computes mean and variance per-sample across all channels (C). Diagram B shows this pass—normalizing across all channels for a single sample at a specific spatial location (H,W).

(b) Covariance Analysis

Given the simplified BN (demean-only) with scalar batch $\{x_1, x_2, \ldots, x_n\}$:

- $a_i = x_i - \mu$ where $\mu = (1/n)\sum_j x_j$
- $y_i = \gamma a_i + \beta$

Hallucinated "covariance analysis"

Covariance:
$\mathrm{Cov}(x_i, y_i) = \gamma\sigma^2$
where $\sigma^2 = (1/n)\sum_j (x_j - \mu)^2$ is the batch variance.

As $n \to \infty$:
$\mathrm{Cov}(x_i, y_i)$ converges to $\gamma\cdot\mathrm{Var\_population}(x)$, i.e., γ multiplied by the true variance of the underlying data distribution (assuming bounded i.i.d. samples).

This model only extracts text from images, switch to K1.5 for full visual understanding.

For the second part (b), we were supposed to calculate dL/dx_i in terms of dL/dy_j for j = 1, ..., n and gamma and beta as needed. Remember y_i = \gamma x_i^{hat} + \beta. Then analyze this as n -> infinity

Batch Normalization Gradient γ Times Individual Minus Batch Mean

Here is the corrected solution for part (b):

(b) Gradient Computation and Asymptotic Analysis

Given a simplified batch normalization (demean-only) with batch $[x_1, x_2, \ldots, x_n]$:

Had to reprompt and restate the question.

Forward Pass:

- $\mu = \frac{1}{n}\sum_{j=1}^n x_j$
- $a_i = x_i - \mu$
- $y_i = \gamma a_i + \beta = \gamma(x_i - \mu) + \beta$

Derivative Computation:

Using the chain rule, $\frac{\partial L}{\partial x_i} = \sum_{j=1}^n \frac{\partial L}{\partial y_j}\frac{\partial y_j}{\partial x_i}$.

For $j = i$:

$$\frac{\partial y_i}{\partial x_i} = \gamma\left(1 - \frac{\partial \mu}{\partial x_i}\right) = \gamma\left(1 - \frac{1}{n}\right)$$

For $j \neq i$:

$$\frac{\partial y_j}{\partial x_i} = \gamma\left(0 - \frac{\partial \mu}{\partial x_i}\right) = -\frac{\gamma}{n}$$

Let $G_j = \frac{\partial L}{\partial y_j}$. Then:

$$\frac{\partial L}{\partial x_i} = \gamma\left(1 - \frac{1}{n}\right)G_i - \frac{\gamma}{n}\sum_{j\neq i} G_j$$

Combining terms:

$$\frac{\partial L}{\partial x_i} = \gamma\left(G_i - \frac{1}{n}\sum_{j=1}^n G_j\right) = \gamma(G_i - \bar{G})$$

where $\bar{G}$ is the batch mean of the upstream gradients.

As $n \to \infty$:

Assuming bounded, i.i.d. samples and gradients:

1. **Forward behavior:** $\mu \to \mathbb{E}[x]$ (population mean), so $y_i \to \gamma(x_i - \mathbb{E}[x]) + \beta$
2. **Backward behavior:** By the Law of Large Numbers, $\bar{G} = \frac{1}{n}\sum_{j=1}^n G_j \to \mathbb{E}\left[\frac{\partial L}{\partial y}\right]$ (expected gradient)

The gradient expression becomes:

$$\frac{\partial L}{\partial x_i} \to \gamma\left(\frac{\partial L}{\partial y_i} - \mathbb{E}\left[\frac{\partial L}{\partial y}\right]\right)$$

Key Implications:

- The gradient is **demeaned** by the expected gradient rather than the batch mean
- The $1/n$ factor ensures the second term vanishes relative to individual gradients as n grows
- Gradient variance stabilizes at $\gamma^2\cdot\mathrm{Var}\left(\frac{\partial L}{\partial y}\right)$
- Computation becomes more stable with larger batch sizes as the population mean is more accurately estimated

What is the value of that gradient as n approaches infinity? Don't use expectations

As $n \to \infty$, the exact (non-asymptotic) expression we already derived is

$$\frac{\partial L}{\partial x_i} = \gamma(G_i - \bar{G}), \qquad \bar{G} = \frac{1}{n}\sum_{j=1}^n G_j.$$

When the batch size grows without bound, the **empirical mean** $\bar{G}$ itself becomes a **deterministic constant** (the population mean of the gradient random variable), so for every fixed sample index i the gradient converges **point-wise** to

$$\frac{\partial L}{\partial x_i} \xrightarrow{n\to\infty} \gamma(G_i - \mu_G)$$

where μ_G is simply the limiting value of the sample mean $\bar{G}$.

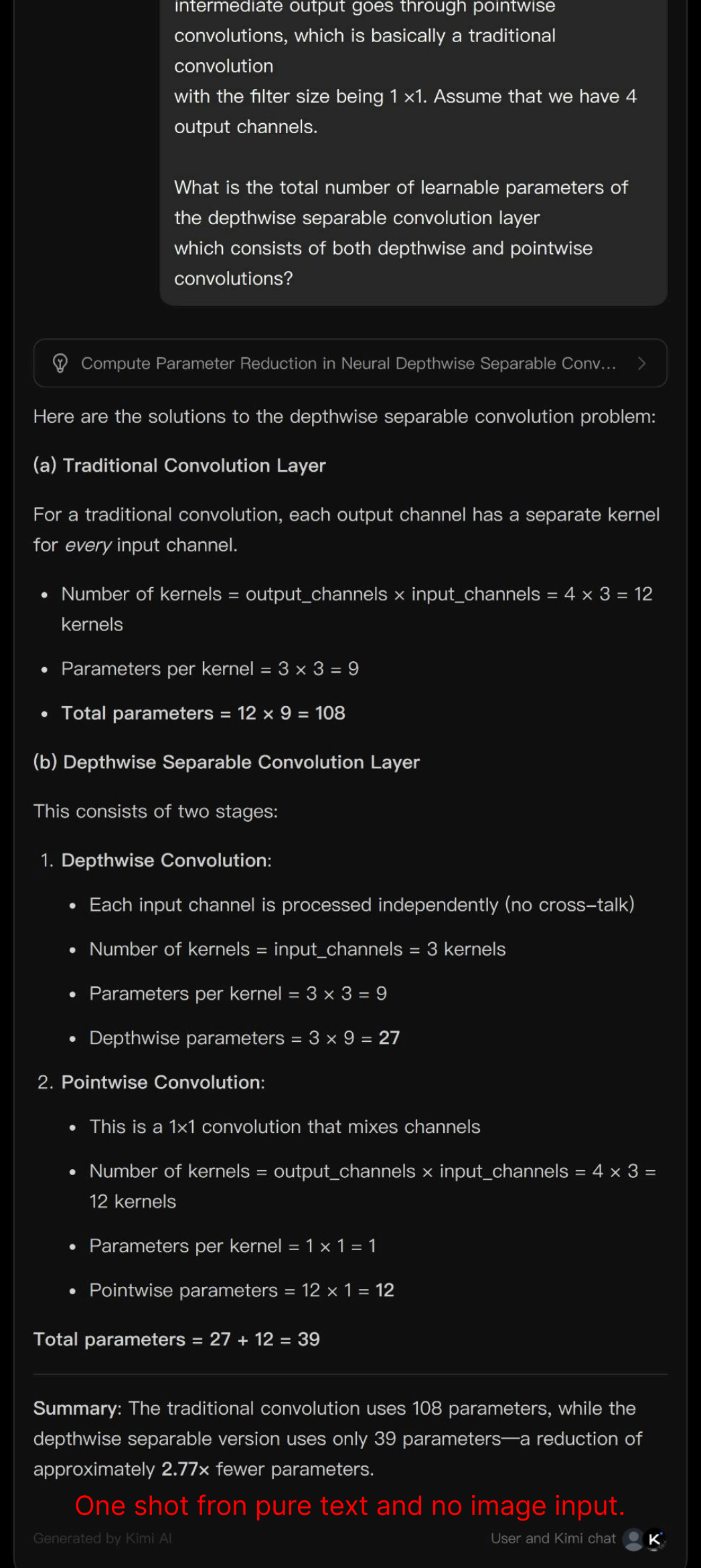
No expectation notation is used—μ_G is the **finite, fixed number** that the running average $\bar{G}$ settles to as the batch size increases.

Got the limit wrong by including the extra sample mean of gradients.

Generated by Kimi AI

User and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents



KIMI

Solve this deep learning problem:

3. Depthwise Separable Convolutions
Depthwise separable convolutions are a type of convolutional operation used in deep learning for image processing tasks. Unlike traditional convolutional operations, which perform both spatial and channel-wise convolutions simultaneously, depthwise separable convolutions decompose the convolution operation into two separate operations: depthwise convolution and pointwise convolution. This can be viewed as a low-rank approximation to a traditional convolution. For simplicity, throughout this problem, we will ignore biases while counting learnable parameters.
(a) Suppose the input is a three-channel 224 ×224-resolution image, the kernel size of the convolutional layer is 3 ×3, and the number of output channels is 4.
What is the number of learnable parameters in the traditional convolution layer?
(b) Depthwise separable convolution consists of two parts: depthwise convolutions (Fig.2) followed by pointwise convolutions (Fig.3). Suppose the input is still a three-channel 224 ×224-resolution image. The input first goes through depthwise convolutions, where the number of output channels is the same as the number of input channels, and there is no "cross talk" between different channels. Then, this intermediate output goes through pointwise convolutions, which is basically a traditional convolution with the filter size being 1×1. Assume that we have 4 output channels.
What is the total number of learnable parameters of the depthwise separable convolution layer which consists of both depthwise and pointwise convolutions?

Compute Parameter Reduction in Neural Depthwise Separable Conv...

Here are the solutions to the depthwise separable convolution problem:

(a) Traditional Convolutional Layer

For a traditional convolution, each output channel has a separate kernel for every input channel.

- Number of kernels = output_channels × input_channels = 4 × 3 = 12 kernels
- Parameters per kernel = 3 × 3 = 9
- **Total parameters = 12 × 9 = 108**

(b) Depthwise Separable Convolution Layer

This consists of two stages:

1. **Depthwise Convolution:**
   - Each input channel is processed independently (no cross-talk)
   - Number of kernels = input_channels = 3 kernels
   - Parameters per kernel = 3 × 3 = 9
   - Depthwise parameters = 3 × 9 = **27**
2. **Pointwise Convolution:**
   - This is a 1×1 convolution that mixes channels
   - Number of kernels = output_channels × input_channels = 4 × 3 = 12 kernels
   - Parameters per kernel = 1 × 1 = 1
   - Pointwise parameters = 12 × 1 = **12**

Total parameters = 27 + 12 = 39

Summary: The traditional convolution uses 108 parameters, while the depthwise separable version uses only 39 parameters—a reduction of approximately 2.77× fewer parameters.

One shot from pure text and no image input.

Generated by Kimi AI

User and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents

KIMI

Solve these deep learning problems.

Asked only first part

Explored Expected Loss with Weighted Norm Regularization Terms

Solution to Dropout Regularization Derivation

We'll derive equation (5) from equation (2) by carefully computing the expectation. Let $X \in \mathbb{R}^{n\times d}$ be the data matrix, $y \in \mathbb{R}^n$ the target vector, $w \in \mathbb{R}^d$ the weight vector, and $R \in \{0,1\}^{n\times d}$ a random matrix with i.i.d. entries $R_{ij} \sim \mathrm{Bernoulli}(p)$.

Step 1: Expand the Expectation

Starting from the dropout objective:

$$C(w) = \mathbb{E}_R\left[\|y - (R\odot X)w\|^2\right]$$

Expand the squared norm:

$$\|y - (R\odot X)w\|^2 = y^Ty - 2y^T(R\odot X)w + w^T(R\odot X)^T(R\odot X)w$$

Taking expectation:

$$C(w) = y^Ty - 2y^T\mathbb{E}_R[R\odot X]w + w^T\mathbb{E}_R[(R\odot X)^T(R\odot X)]w$$

Step 2: Compute $\mathbb{E}_R[R\odot X]$

Since $R_{ij} \sim \mathrm{Bernoulli}(p)$, we have $\mathbb{E}[R_{ij}] = p$ for all i, j. Therefore:

$$\mathbb{E}_R[(R\odot X)_{ij}] = \mathbb{E}[R_{ij}]X_{ij} = pX_{ij}$$

Thus:

$$\mathbb{E}_R[R\odot X] = pX$$

Step 3: Compute $\mathbb{E}_R[(R\odot X)^T(R\odot X)]$

Let $M = R\odot X$. The (j, k) entry of M^TM is:

$$(M^TM)_{jk} = \sum_{i=1}^n M_{ij}M_{ik} = \sum_{i=1}^n R_{ij}X_{ij}R_{ik}X_{ik}$$

Taking expectation and using properties of Bernoulli variables:

Case 1: Diagonal entries ($j = k$)

$$\mathbb{E}[(M^TM)_{jj}] = \sum_{i=1}^n X_{ij}^2\,\mathbb{E}[R_{ij}^2] = p\sum_{i=1}^n X_{ij}^2 = p\|X_{\cdot j}\|^2$$

Case 2: Off-diagonal entries ($j \neq k$)

Since R_{ij} and R_{ik} are independent:

$$\mathbb{E}[(M^TM)_{jk}] = \sum_i X_{ij}X_{ik}\,\mathbb{E}[R_{ij}]\,\mathbb{E}[R_{ik}] = p^2\sum_i X_{ij}X_{ik} = p^2(X^TX)_{jk}$$

Combining both cases, we can write:

$$\mathbb{E}_R[(R\odot X)^T(R\odot X)] = p^2X^TX + p(1-p)D$$

where D is a diagonal matrix with $D_{jj} = \|X_{\cdot j}\|^2$ (the squared ℓ_2-norm of the j-th column of X).

Verification: For $j = k$, the expression gives $p^2\|X_{\cdot j}\|^2 + p(1-p)\|X_{\cdot j}\|^2 = p\|X_{\cdot j}\|^2$. For $j \neq k$, it gives $p^2(X^TX)_{jk}$ since D contributes only to diagonal entries.

Step 4: Assemble the Final Expression

Substituting back:

$$C(w) = y^Ty - 2py^TXw + w^T\left[p^2X^TX + p(1-p)D\right]w$$

Separate into two terms:

$$\underbrace{y^Ty - 2py^TXw + p^2w^TX^TXw}_{\|y-pXw\|^2} + \underbrace{p(1-p)w^TDw}_{\text{regularization}}$$

The first term is:

$$\|y - pXw\|^2 = (y - pXw)^T(y - pXw)$$

For the second term, let Γ be a diagonal matrix with $\Gamma_{jj} = \|X_{\cdot j}\|$. Then:

$$\|\Gamma w\|^2 = w^T\Gamma^T\Gamma w = w^TDw$$

since $\Gamma^T\Gamma = D$ (diagonal matrices commute).

Final Result

Correct answer + proof

Therefore:

$$C(w) = \|y - pXw\|^2 + p(1-p)\|\Gamma w\|^2$$

where Γ is a diagonal matrix whose j-th diagonal entry is $\|X_{\cdot j}\|$, the ℓ_2-norm of the j-th column of the training matrix X. This matches equation (5) and reveals that **dropout implicitly performs L_2 regularization weighted by the squared column norms of X**, while scaling the effective input by p.

This model only extracts text from images, switch to K1.5 for full visual understanding.

(b) How should we transform the ˆw we learn using (5) (i.e. with dropout) to get something that looks a solution to the traditionally regularized problem (3)? (Hint: This is related to how we adjust weights learned using dropout training for using them at inference time. PyTorch by default does this adjustment during training itself, but here, we are doing dropout slightly differently with no adjustments during training.)

Scale Dropout Weights by Keep Probability to Match Traditional Re...

Part (b): Transformation of Dropout-Learned Weights

To transform the weights $\hat{w}$ learned via dropout (objective (5)) into a solution that resembles the traditionally regularized problem (3), **scale the weight vector by the keep probability** p:

$$w_{\text{transformed}} = p\cdot\hat{w}$$

Correct answer

Why This Works:

1. **Training vs. Inference Mismatch:** During dropout training, the effective input is $(R\odot X)$ where features are kept with probability p. This creates an expected scaling of p in the data term (pXw in (5)). At inference, we use the full, unmasked input X.
2. **Matching Expected Activations:** The learned weights $\hat{w}$ expect inputs to be scaled by p. To maintain the same expected forward pass at inference (without dropout), we scale the weights by p. This ensures:

$$X\cdot(p\hat{w}) = pX\hat{w}$$

which matches the expectation of the dropout training forward pass.

3. **Connection to Traditional Regularization:** When you substitute $w = p\hat{w}$ into the dropout objective (5) and simplify, you get:

$$\|y - X(p\hat{w})\|^2 + \frac{1-p}{p}\|\Gamma(p\hat{w})\|^2 = \|y - Xw_{\text{transformed}}\|^2 + \frac{1-p}{p}\|\Gamma w$$

This has the same form as the traditional problem (3), but with a rescaled regularization constant. The key insight is that $w_{\text{transformed}}$ is directly compatible with the standard (non-dropout) prediction equation.

4. **PyTorch Comparison:** This is equivalent to the **weight scaling approximation** used in vanilla dropout. PyTorch's inverted dropout performs this scaling during training (by multiplying kept activations by $1/p$), eliminating the need for adjustment at inference. Since this problem uses vanilla dropout (no training-time scaling), you must apply the p scaling to the weights at test time.

In short: Multiply your dropout-trained weights by p before using them for prediction on full (non-dropout) inputs.

Solve the problem in this image...

Transform Tikhonov Regularization to Ridge Regression via Data Ma...

Part (c): Transforming to Classical Ridge Regression

Given the Tikhonov-regularized problem:

$$C(w) = \|y - Xw\|^2 + \|\Gamma w\|^2 \quad (3)$$

where Γ is an invertible diagonal matrix (from dropout, $\Gamma_{jj} = \|X_{\cdot j}\|$), we can transform it to classical ridge regression:

$$C(\tilde{w}) = \|y - \tilde{X}\tilde{w}\|^2 + \lambda\|\tilde{w}\|^2 \quad (6)$$

Change of Variables

Define the linear transformation:

$$u = \Gamma w \implies w = \Gamma^{-1}u$$

Substituting into equation (3):

$$C(u) = \|y - X\Gamma^{-1}u\|^2 + \|u\|^2$$

This is now in the classical ridge regression form where:

- **Changed data matrix:** $\tilde{X} = X\Gamma^{-1}$
- **New weight vector:** u (you can rename this back to $\tilde{w}$)
- **Regularization parameter:** $\lambda = 1$

Explicit Answer

The changed data matrix is:

$$\tilde{X} = X\Gamma^{-1}$$

Correct answer

This transformation works because Γ is invertible (its diagonal entries are the non-zero column norms). Geometrically, you're rescaling each feature dimension by its column norm before applying standard L_2 regularization.

This model only extracts text from images, switch to K1.5 for full visual understanding.

Continuing the previous problem with the further assumption that Γ is a diagonal invertible matrix with the j-th diagonal entry proportional to the norm of the j-th column in X, what can you say about the norms of the columns of the effective training matrix X̃ and speculate briefly on the relationship between dropout and batch-normalization.

Dropout and Batch Normalization as Adaptive Regularization for Sca...

Part (d): Column Norms and Relationship to Batch Normalization

Column Norms of $\tilde{X}$:

The effective training matrix $\tilde{X} = X\Gamma^{-1}$ has unit-norm columns.

Proof: For each column j:

- $\tilde{X}_{\cdot j} = X_{\cdot j}\cdot(\Gamma^{-1})_{jj} = X_{\cdot j}/\|X_{\cdot j}\|$
- $\|\tilde{X}_{\cdot j}\| = \|X_{\cdot j}\|/\|X_{\cdot j}\| = 1$

Thus, all columns are normalized to have unit ℓ_2-norm.

Speculation on Dropout vs. Batch Normalization:

This reveals a deep connection: **Dropout implicitly performs a form of feature normalization**, much like batch normalization:

1. **Adaptive Regularization:** Dropout's penalty term $p(1-p)\|\Gamma w\|^2$ scales with feature norms – large-norm features are regularized more aggressively. This encourages the network to be invariant to feature scaling without explicit normalization layers.
2. **Implicit Whitening:** By transforming to $\tilde{X}$ with unit columns, dropout creates an effective learning problem where features are on equal footing, similar to how batch normalization standardizes activations.
3. **Redundancy:** Both techniques combat internal covariate shift. When batch normalization already stabilizes feature distributions, dropout's implicit scaling provides less additional benefit, which may explain why modern architectures often use batch normalization instead of (or with reduced) dropout.

Correct answer (used c=1 from earlier)

Generated by Kimi AI

User and Kimi chat

Scan QR code to chat with Kimi
Ask questions · Web search · Read documents